

Date:
Student Name:
Student ID:

AI Workshop #4

Predictive Analytics with Ensemble Learning

4.1. Introduction

In this workshop, we will learn about ensemble learning and how to use it for predictive analytics.

By the end of this workshop, you will have a better understanding of these topics:

- Decision trees and decision trees classifiers
- Learning models with ensemble learning
- Random forests and extremely random forests
- Confidence measure estimation of predictions
- Dealing with class imbalance
- Finding optimal training parameters using grid search
- Computing relative feature importance
- Traffic prediction using the extremely random forests regressor

Let's begin with **decision trees**. Firstly, what are they?

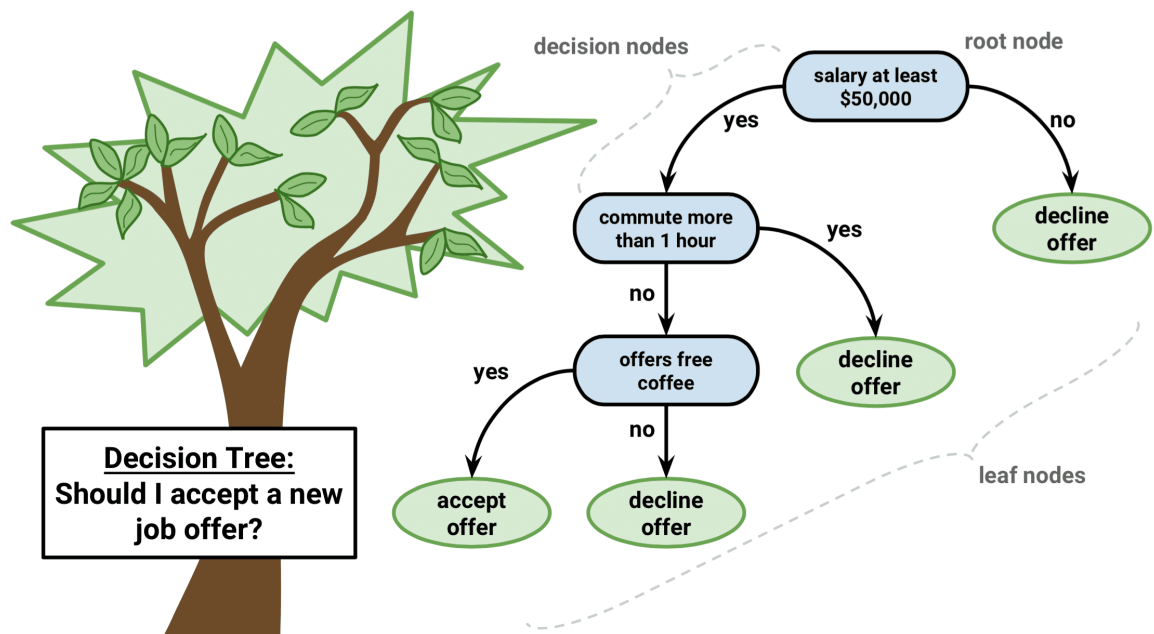
4.2 What are decision trees?

A decision tree is a way to partition a dataset into distinct branches.

The branches or partitions are then traversed to make simple decisions. Decision trees are produced by training algorithms, which identify how to split the data in an optimal way.

The decision process starts at the root node at the top of the tree. Each node in the tree is a decision rule. Algorithms construct these rules based on the relationship between the input data and the target labels in the training data. The values in the input data are utilized to estimate the value of the output.

The following shows a Decision Tree for the decision of Job Offering.



Now that we understand the basic concept behind decision trees, the next concept to understand is how the trees are automatically constructed.

We need algorithms that can construct the optimal tree based on the data. In order to understand it, we need to understand the concept of entropy. In this context, entropy refers to information entropy and not thermodynamic entropy. Information entropy is basically a measure of uncertainty.

One of the main goals of a decision tree is to reduce uncertainty as we move from the root node towards the leaf nodes.

Building a decision tree classifier

Let's see how to build a classifier using decision trees in Python.

1. Create a new Python file and import the following packages:

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from utilities import visualize_classifier
```

We will be using the data in the **data_decision_trees.txt** file that's provided to you. In this file, each line contains comma-separated values. The first two values correspond to the input data and the last value corresponds to the target label.

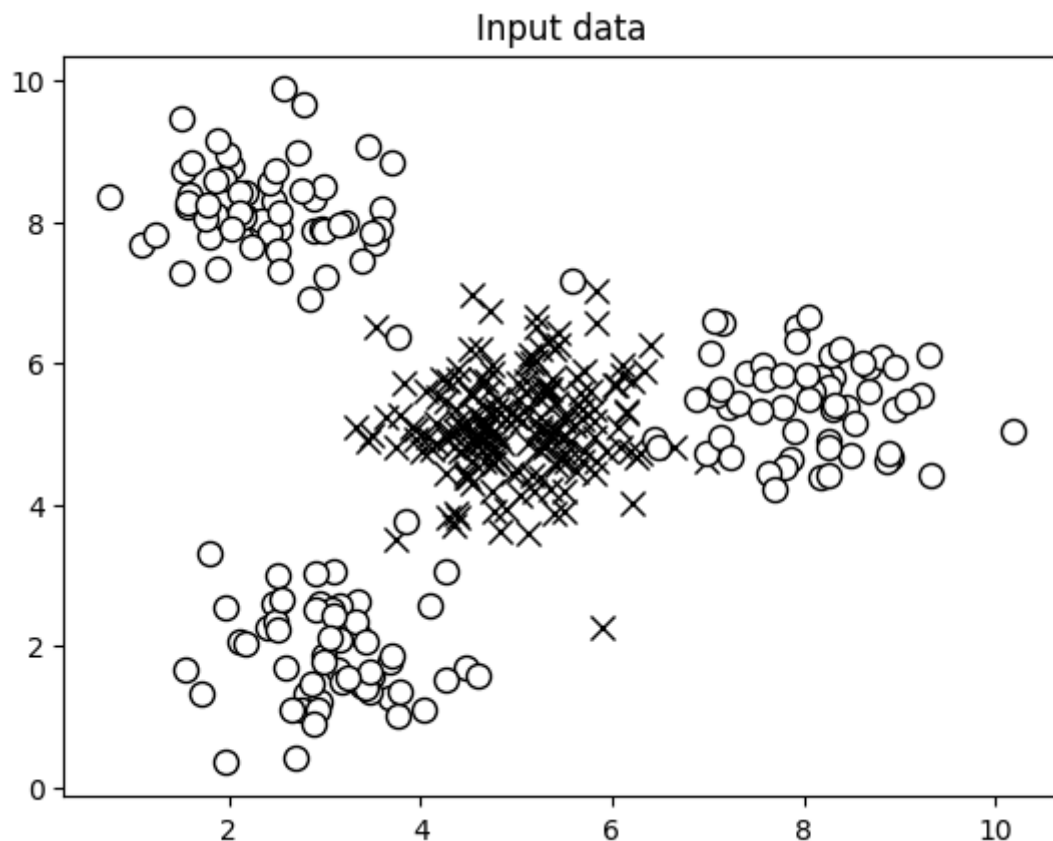
2. Let's load the data from that file:


```
In [7]: # Visualize input data
plt.figure()
plt.scatter(class_0[:, 0], class_0[:, 1], s=75, facecolors='black', edgecolor='black')
plt.scatter(class_1[:, 0], class_1[:, 1], s=75, facecolors='white', edgecolor='black')
plt.title('Input data')
```

C:\Users\Raymond Lee\AppData\Local\Temp\ipykernel_14304\3721073358.py:3: UserWarning: You passed a edgecolor/edgecolors ('black') for an unfilled marker ('x'). Matplotlib is ignoring the edgecolor in favor of the facecolor. This behavior may change in the future.

```
plt.scatter(class_0[:, 0], class_0[:, 1], s=75, facecolors='black', edgecolor='black', linewidth=1, marker='x')
```

Out[7]: Text(0.5, 1.0, 'Input data')

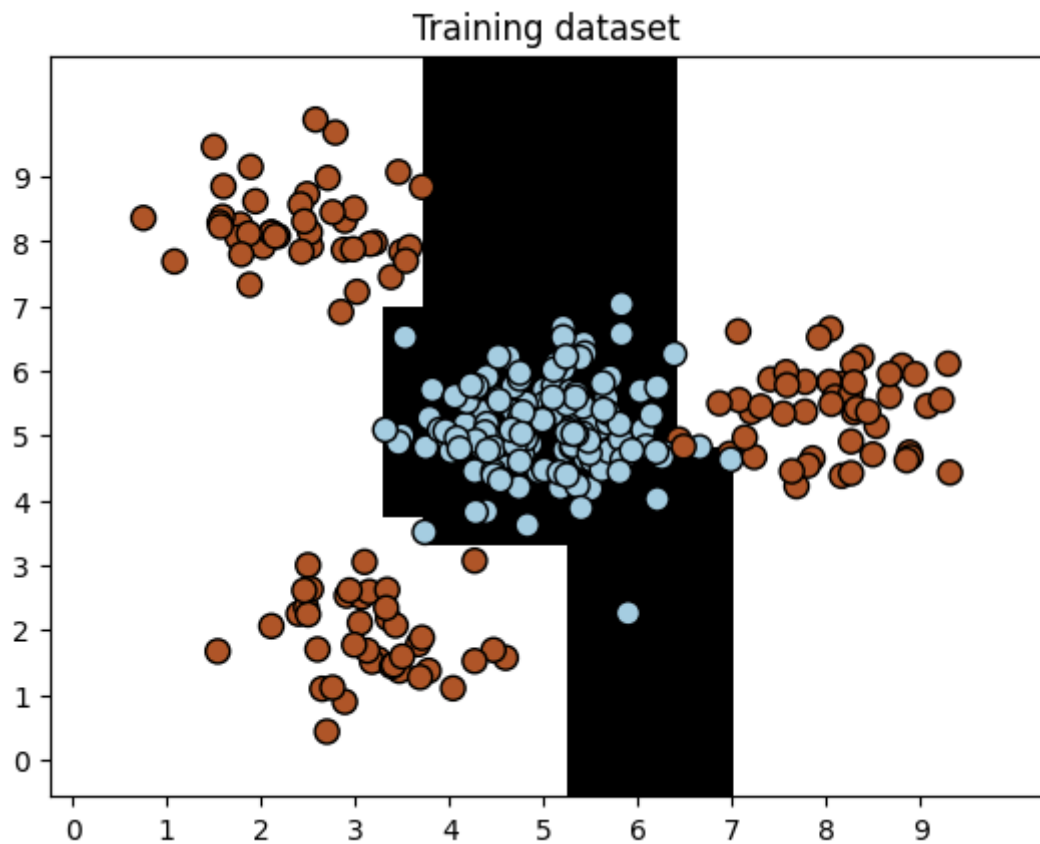


6. We need to split the data into training and testing datasets:

```
In [8]: # Split data into training and testing datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, ra
```

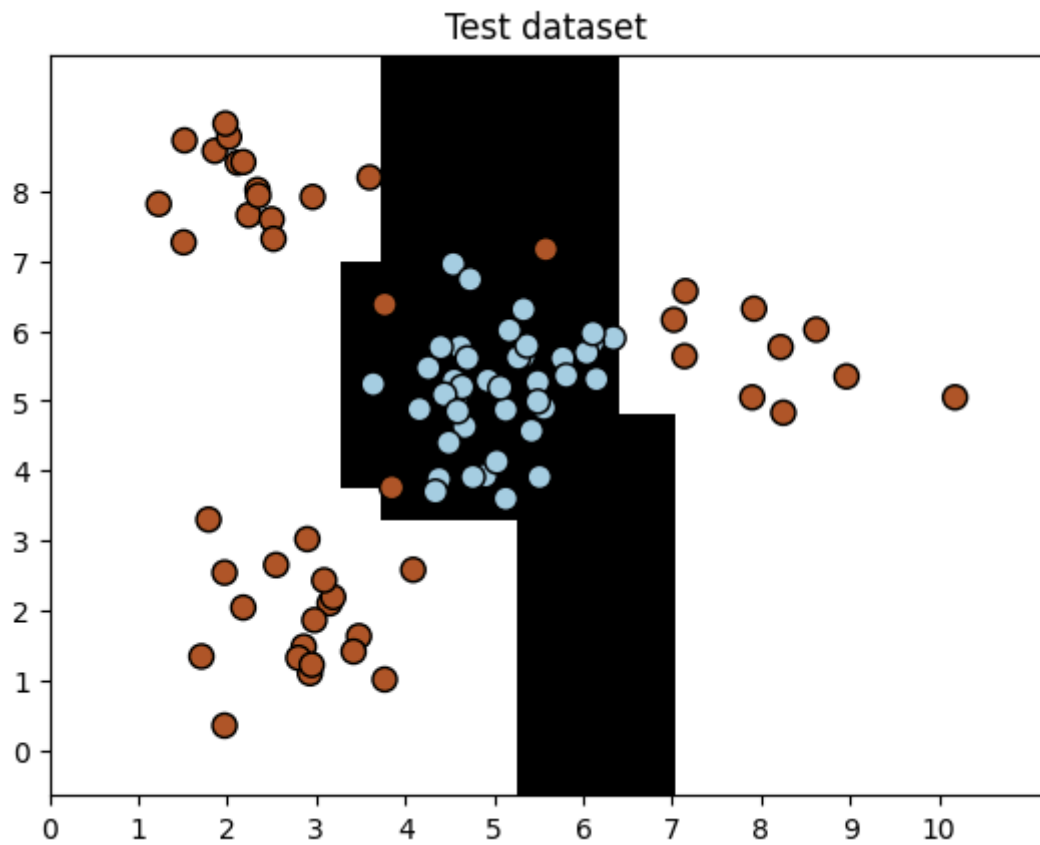
7. Create, build, and visualize a decision tree classifier based on the training dataset. The `random_state` parameter refers to the seed used by the random number generator required for the initialization of the decision tree classification algorithm. The `max_depth` parameter refers to the maximum depth of the tree that we want to construct:

```
In [9]: # Decision Trees classifier
params = {'random_state': 0, 'max_depth': 4}
classifier = DecisionTreeClassifier(**params)
classifier.fit(X_train, y_train)
visualize_classifier(classifier, X_train, y_train, 'Training dataset')
```



8. Compute the output of the classifier on the test dataset and visualize it:

```
In [10]: y_test_pred = classifier.predict(X_test)
visualize_classifier(classifier, X_test, y_test, 'Test dataset')
```



9. Evaluate the performance of the classifier by printing the classification report:

```
In [11]: # Evaluate classifier performance
class_names = ['Class-0', 'Class-1']
print("\n" + "#" * 40)
print("\nClassifier performance on training dataset\n")
print(classification_report(y_train, classifier.predict(X_train), target_names=class_names))
print("#" * 40 + "\n")

print("#" * 40)
print("\nClassifier performance on test dataset\n")
print(classification_report(y_test, y_test_pred, target_names=class_names))
print("#" * 40 + "\n")

plt.show()
```

#####

Classifier performance on training dataset

	precision	recall	f1-score	support
Class-0	0.99	1.00	1.00	137
Class-1	1.00	0.99	1.00	133
accuracy			1.00	270
macro avg	1.00	1.00	1.00	270
weighted avg	1.00	1.00	1.00	270

#####

#####

Classifier performance on test dataset

	precision	recall	f1-score	support
Class-0	0.93	1.00	0.97	43
Class-1	1.00	0.94	0.97	47
accuracy			0.97	90
macro avg	0.97	0.97	0.97	90
weighted avg	0.97	0.97	0.97	90

#####



The performance of a classifier is characterized by **precision, recall, and F1-scores**. **Precision** refers to the accuracy of the classification, and **recall** refers to the number of items that were retrieved as a percentage of the overall number of items that were supposed to be retrieved.

A good classifier will have high precision and high recall, but there is usually a trade-off between the two. Hence, we have **F1-score** to characterize that. F1 score is the harmonic mean of precision and recall, which gives it a good balance between precision and recall values.

Decision trees are an example of using a single model to make a prediction. It is possible to sometimes create stronger models and better predictions by combining and aggregating the results of multiple models.



Workshop 4.1 : Decision Tree with Modified Dataset

1. By modifying the classification of 20% of the training data, try to show the decision tree classification result and compared with the previous results. What is your findings.
2. Try to modify 20% of the testset classification as well and compare the results.

4.3 What is ensemble learning?

Ensemble learning involves building multiple models and then combining them in such a way that it produces better results than what the models could produce individually.

These individual models can be classifiers, regressors, or other models.

Ensemble learning is used extensively across multiple fields, including data classification, predictive modeling, and anomaly detection.

So why use ensemble learning? In order to gain understanding, let's use a real-life example. You want to buy a new TV, but you don't know what the latest models are. Your goal is to get the best value for your money, but you don't have enough knowledge on this topic to make an informed decision. When you must decide about something like this, you might get the opinions of multiple experts in the domain. This will help you make the best decision. Often, instead of relying on one opinion, you can decide by combining the individual decisions of those experts. Doing this minimizes the possibility of making a wrong or suboptimal decision.

Building learning models with ensemble learning

While selecting a model, a commonly used procedure is to choose the one with the smallest error on the training dataset. The problem with this approach is that it will not always work. The model might become biased or overfit the training data. Even when training the model using cross-validation, it can perform poorly on unknown data.

A reason why ensemble learning models are effective is because they reduce the overall risk of making a poor model selection. This enables it to train in a diverse manner and then perform well on unknown data. When a model is built using ensemble learning, the individual models need to exhibit some diversity. This allows them to capture various nuances in the data; hence the overall model becomes more accurate.

The diversity is achieved by using different training parameters for each individual model. This allows individual models to generate different decision boundaries for training data. This means that each model will use different rules to make an inference, which is a powerful way of validating the result. If there is agreement among the models, this increases the confidence in the prediction.

A special type of an ensemble learning is when you combine decision trees into an ensemble. These models are usually known as random forests and extremely random forests, which we'll describe in the coming sections.

What are random forests and extremely random forests?

A **random forest** is an instance of ensemble learning where individual models are constructed using decision trees. This ensemble of decision trees is then used to predict the output value. We use a random subset of training data to construct each decision tree.

This will ensure diversity among various decision trees. In the first section, we discussed that one of the most important attributes when building good ensemble learning models is that we ensure that there is diversity among individual models.

One of the advantages of random forests is that they do not overfit. Overfitting is a frequent problem in machine learning. Overfitting is more likely with nonparametric and nonlinear models that have more flexibility when learning a target function. By constructing a diverse set of decision trees using various random subsets, we ensure that the model does not overfit the training data. During the construction of the tree, the nodes are split successively, and the best thresholds are chosen to reduce the entropy at each level. This split doesn't consider all the features in the input dataset. Instead, it chooses the best split among the random subset of the features that is under consideration. Adding this randomness tends to increase the bias of the random forest, but the variance decreases because of averaging. Hence, we end up with a robust model.

Extremely random forests take randomness to the next level. Along with taking a random subset of features, the thresholds are chosen randomly as well. These randomly generated thresholds are chosen as the splitting rules, which reduce the variance of the model even further. Hence, the decision boundaries obtained using extremely random forests tend to be smoother than the ones obtained using random forests. Some implementations of extremely

Building random forest and extremely random forest classifiers

Let's see how we can build a classifier based on random forests and extremely random forests. The way to construct both classifiers is very similar, so an input flag is used to specify which classifier needs to be built.

1. Create a new Python file and import the following packages:

```
In [12]: import argparse

import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier
from sklearn.metrics import classification_report

from utilities import visualize_classifier
```

2. Define an argument parser for Python so that we can take the classifier type as an input parameter. Depending on this parameter, we can construct a random forest classifier or an extremely random forest classifier:

```
In [13]: # Argument parser
def build_arg_parser():
    parser = argparse.ArgumentParser(description='Classify data using \
        Ensemble Learning techniques')
    parser.add_argument('--classifier-type', dest='classifier_type',
        required=True, choices=['rf', 'erf'], help="Type of classifier \
        to use; can be either 'rf' or 'erf'")
    return parser
```

3. Define the main function and parse the input arguments:

```
if __name__ == '__main__':
    # Parse the input arguments
    args = build_arg_parser().parse_args()
    classifier_type = args.classifier_type
```

4. We will be using the data from the data_random_forests.txt file that is provided to you. Each line in this file contains comma-separated values. The first two values correspond to the input data and the last value corresponds to the target label. We have three distinct classes in this dataset. Let's load the data from that file:

```
# Load input data
input_file = 'data_random_forests.txt'
data = np.loadtxt(input_file, delimiter=',')
X, y = data[:, :-1], data[:, -1]
```

5. Separate the input data into three classes:

```
# Separate input data into three classes based on labels
class_0 = np.array(X[y==0])
class_1 = np.array(X[y==1])
class_2 = np.array(X[y==2])
```

6. Let's visualize the input data:

```
# Visualize input data
```

```
plt.figure()
plt.scatter(class_0[:, 0], class_0[:, 1], s=75, facecolors='white',
            edgecolors='black', linewidth=1, marker='s')
plt.scatter(class_1[:, 0], class_1[:, 1], s=75, facecolors='white',
            edgecolors='black', linewidth=1, marker='o')
plt.scatter(class_2[:, 0], class_2[:, 1], s=75, facecolors='white',
            edgecolors='black', linewidth=1, marker='^')
plt.title('Input data')
```

7. Split the data into training and testing datasets:

```
# Split data into training and testing datasets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=5)
```

8. Define the parameters to be used when we construct the classifier. The **n_estimators** parameter refers to the number of trees that will be constructed. The **max_depth** parameter refers to the maximum number of levels in each tree. The **random_state** parameter refers to the seed value of the random number generator needed to initialize the random forest classifier algorithm:

```
# Ensemble Learning classifier
params = {'n_estimators': 100, 'max_depth': 4, 'random_state': 0}
```

9. Depending on the input parameter, we either construct a random forest classifier or an extremely random forest classifier:

```
if classifier_type == 'rf':
    classifier = RandomForestClassifier(**params)
else:
    classifier = ExtraTreesClassifier(**params)
```

10. Train and visualize the classifier:

```
classifier.fit(X_train, y_train)
visualize_classifier(classifier, X_train, y_train, 'Training dataset')
```

11. Compute the output based on the test dataset and visualize it:

```
y_test_pred = classifier.predict(X_test)
visualize_classifier(classifier, X_test, y_test, 'Test dataset')
```

12. Evaluate the performance of the classifier by printing the classification report:

```
# Evaluate classifier performance
class_names = ['Class-0', 'Class-1', 'Class-2']
print("\n" + "#" * 40)
print("\nClassifier performance on training dataset\n")
print(classification_report(y_train, classifier.predict(X_train),
target_names=class_names))
```

```

print("#"*40 + "\n")

print("#"*40)
print("\nClassifier performance on test dataset\n")
print(classification_report(y_test, y_test_pred,
target_names=class_names))
print("#"*40 + "\n")

```



Workshop 4.2 : Construct random_forests.py

1. Combine the code fragment of the above and construct the complete random_forests.py program code.
2. Run the code with the random forest classifier using the **rf** flag in the input argument by running the following command:

```
$ python random_forests.py --classifier-type rf
```

3. Save the two figures (if you are successful) into the following for submission.
4. Now run the code with the extremely random forest classifier by using the erf flag in the input argument. Run the following command:

```
$ python random_forests.py --classifier-type erf
```

5. You should see a few figures pop up. Save these figures for submission.
6. If you compare the preceding screenshot with the boundaries obtained from the random forest classifier, you will see that these boundaries are smoother. Why. Please explain.

Estimating the confidence measure of the predictions

If you analyze the output, you will see that the probabilities are printed for each data point. These probabilities are used to measure the confidence values for each class. Estimating the confidence values is an important task in machine learning.

13. In the same Python file, add the following line to define an array of test data points:

```

# Compute confidence
test_datapoints = np.array([[5, 5], [3, 6], [6, 4], [7, 2], [4, 4], [5,
2]])

```

14. The classifier object has an inbuilt method to compute the confidence measure. Let's classify each point and compute the confidence values:

```
print("\nConfidence measure:")
for datapoint in test_datapoints:
    probabilities = classifier.predict_proba([datapoint])[0]
    predicted_class = 'Class-' + str(np.argmax(probabilities))
    print('\nDatapoint:', datapoint)
    print('Predicted class:', predicted_class)
```

15. Visualize the test data points based on classifier boundaries:

```
# Visualize the datapoints
visualize_classifier(classifier, test_datapoints,
[0]*len(test_datapoints),
    'Test datapoints')

plt.show()
```



Workshop 4.3 : Construct random_forests.py + Confidence Measure

1. Combine the code **Random_Forecasts.py** with the above confidence Measure to produce random_forests_Conf.py.
2. Run the code with **erf** flag. Save the output for submission.

4.4 Dealing with class imbalance

A classifier is only as good as the data that is used for training.

A common problem faced in the real world is issues with data quality. For a classifier to perform well, it needs to see an equal number of points for each class.

But when data is collected in the real world, it's not always possible to ensure that each class has the exact same number of data points.

If one class has 10 times the number of data points than another class, then the classifier tends to get biased towards the more numerous class.

Hence, we need to make sure that we account for this imbalance algorithmically.

1. Create a new Python file and import the following packages:

```
import sys

import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

from utilities import visualize_classifier
```

2. We will use the data in the file **data_imbalance.txt** for our analysis. Let's load the data. Each line in this file contains comma-separated values. The first two values correspond to the input data, and the last value corresponds to the target label. We have two classes in this dataset. Let's load the data from that file:

```
# Load input data
input_file = 'data_imbalance.txt'
data = np.loadtxt(input_file, delimiter=',')
X, y = data[:, :-1], data[:, -1]
```

3. Separate the input data into two classes:

```
# Separate input data into two classes based on labels
class_0 = np.array(X[y==0])
class_1 = np.array(X[y==1])
```

4. Visualize the input data using a scatter plot:

```
# Visualize input data
plt.figure()
plt.scatter(class_0[:, 0], class_0[:, 1], s=75, facecolors='black',
            edgecolors='black', linewidth=1, marker='x')
plt.scatter(class_1[:, 0], class_1[:, 1], s=75, facecolors='white',
            edgecolors='black', linewidth=1, marker='o')
plt.title('Input data')
```

5. Split the data into training and testing datasets:

```
# Split data into training and testing datasets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=5)
```

6. Next, we define the parameters for the extremely random forest classifier.

```
# Extremely Random Forests classifier
params = {'n_estimators': 100, 'max_depth': 4, 'random_state': 0}
if len(sys.argv) > 1:
    if sys.argv[1] == 'balance':
        params = {'n_estimators': 100, 'max_depth': 4, 'random_state': 0,
'class_weight': 'balanced'}
    else:
        raise TypeError("Invalid input argument; should be 'balance'")
```

7. Build, train, and visualize the classifier using training data:

```
classifier = ExtraTreesClassifier(**params)
classifier.fit(X_train, y_train)
visualize_classifier(classifier, X_train, y_train, 'Training dataset')
```

8. Predict the output for the test dataset and visualize the output:

```
y_test_pred = classifier.predict(X_test)
visualize_classifier(classifier, X_test, y_test, 'Test dataset')
```

9. Compute the performance of the classifier and print the classification report:

```
# Evaluate classifier performance
class_names = ['Class-0', 'Class-1']
print("\n" + "#" * 40)
print("\nClassifier performance on training dataset\n")
print(classification_report(y_train, classifier.predict(X_train),
target_names=class_names))
print("#" * 40 + "\n")

print("#" * 40)
print("\nClassifier performance on test dataset\n")
print(classification_report(y_test, y_test_pred, target_names=class_names))
print("#" * 40 + "\n")

plt.show()
```



Workshop 4.4 : Construct class_imbalance.py

1. Combine the code fragment of the above and construct the complete **class_imbalance.py** program code.

2. Run the code and you should see the two graphics: Input Data and Test Dataset which shows the classifier boundary for the test dataset. Save and submit these TWO figures.
3. You should see that the boundary was not able to capture the actual boundary between the two classes. The black patch near the top represents the boundary. Why? Please explain.
4. Finally you will see the classifier performance table. Save and submit it.
5. You see a warning because the values are 0 in the first row, which leads to a divide-by-zero error (ZeroDivisionError exception) when we compute the f1-score. Run the code using the ignore flag so that you do not see the divide-by-zero warning:

```
$ python -W ignore class_imbalance.py
```

6. Now, if you want to account for class imbalance, run it with the balance flag:

```
$ python class_imbalance.py balance
```

7. The problem should be solved. Right? Please save and submit the new Classifier Figure and Classifier Performance table.

4.5 Finding optimal training parameters using grid search

When working with classifiers, it is not always possible to know what the best parameters are to use.

It is not efficient to use brute force by checking for all possible combinations manually.

This is where grid search becomes useful. **Grid search** allows us to specify a range of values and the classifier will automatically run various configurations to figure out the best combination of parameters.

Let's see how to do it.

1. Create a new Python file and import the following packages:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report
import grid_search
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

from utilities import visualize_classifier
```

Noted that we use the **grid search** class provided in this directory.

2. We will use the data available in data_random_forests.txt for analysis:

```
# Load input data
input_file = 'data_random_forests.txt'
data = np.loadtxt(input_file, delimiter=',')
```

```
X, y = data[:, :-1], data[:, -1]
```

3. Separate the data into three classes:

```
# Separate input data into three classes based on labels
class_0 = np.array(X[y==0])
class_1 = np.array(X[y==1])
class_2 = np.array(X[y==2])
```

4. Split the data into training and testing datasets:

```
# Split the data into training and testing datasets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=5)
```

5. Specify the grid of parameters for the classifier to test. Usually one parameter is kept constant and the other parameter is varied. Then the inverse is done to figure out the best combination. In this case, we want to find the best values for `n_estimators` and `max_depth`. Let's specify the parameter grid:

```
# Define the parameter grid
parameter_grid = [ {'n_estimators': [100], 'max_depth': [2, 4, 7, 12, 16]},
                   {'max_depth': [4], 'n_estimators': [25, 50, 100, 250]}
                 ]
```

6. Let's define the metrics that the classifier should use to find the best combination of parameters:

```
metrics = ['precision_weighted', 'recall_weighted']
```

7. For each metric, we need to run the grid search, where we train the classifier for a combination of parameters:

```
for metric in metrics:
    print("\n##### Searching optimal parameters for", metric)

    classifier = grid_search.GridSearchCV(
        ExtraTreesClassifier(random_state=0),
        parameter_grid, cv=5, scoring=metric)
    classifier.fit(X_train, y_train)
```

8. Print the score for each parameter combination and the performance report:

```
print("\nGrid scores for the parameter grid:")
means = classifier.cv_results_['mean_test_score']
params = classifier.cv_results_['params']
for mean, param in zip(means, params):
```

```
print("%f with: %r" % (mean, param))
```



Workshop 4.5 : Construct `run_grid_search.py`

1. Combine the code fragment of the above and construct the complete **`run_grid_search.py`** Python program.
2. Run the code (if successful) you will see TWO reports:

Searching optimal parameters for precision_weighted

Grid scores for the parameter grid: 0.849757 with: {'max_depth': 2, 'n_estimators': 100}
....

Searching optimal parameters for recall_weighted

Grid scores for the parameter grid: 0.842963 with: {'max_depth': 2, 'n_estimators': 100}
....

3. Save and submit these two report for submission.
4. One final step: Amend the program code and output the BEST Parameters. What is the Best Parameters.
5. Save and submit the BEST Parameter results.

4.6 Computing relative feature importance

When working with a dataset that contains N-dimensional data points, it must be understood that not all features are equally important. Some are more discriminative than others. If we have this information, we can use it to reduce the dimensionality. This is useful in reducing the complexity and increasing the speed of the algorithm. Sometimes, a few features are completely redundant. Hence, they can be easily removed from the dataset.

We will be using the AdaBoost regressor to compute feature importance. AdaBoost, short for Adaptive Boosting, is an algorithm that's frequently used in conjunction with other machine learning algorithms to improve their performance. In AdaBoost, the training data points are drawn from a distribution to train the current classifier. This distribution is updated iteratively so that the subsequent classifiers get to focus on the more difficult data points. The difficult data points are the ones that are misclassified. This is done by updating the distribution at each step. This will make the data points that were previously misclassified more likely to come up in the next sample dataset that's used for training.

These classifiers are then cascaded, and the decision is taken through weighted majority voting.

1. Create a new Python file and import the following packages:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import AdaBoostRegressor
from sklearn import datasets
from sklearn.metrics import mean_squared_error, explained_variance_score
from sklearn.model_selection import train_test_split
from sklearn.utils import shuffle
```

2. We will use the Boston Housing Dataset provided by the workshop:

```
# Load housing data
import pandas as pd

# Load the dataset
data = pd.read_csv('boston_house_prices.csv')

data_target = np.array(data.MEDV)
data_feature = np.array(data.iloc[:, :-1])

# Display the first few rows of the dataset
# print(data.head())

print(data_target.shape)
print(data_feature.shape)
```

3. Shuffle the data so that we don't bias our analysis:

```
# Shuffle the data
# X, y = shuffle(housing_data.data, housing_data.target, random_state=7)
X, y = shuffle(data_feature, data_target, random_state=7)
```

4. Split the dataset into training and testing:

```
# Split data into training and testing datasets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=7)
```

5. Define and train an AdaBoostRegressor using the decision tree regressor as the individual model:

```
# AdaBoost Regressor model
regressor = AdaBoostRegressor(DecisionTreeRegressor(max_depth=4),
    n_estimators=400, random_state=7)
regressor.fit(X_train, y_train)
```

6. Estimate the performance of the regressor:

```
# Evaluate performance of AdaBoost regressor
y_pred = regressor.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
evs = explained_variance_score(y_test, y_pred )
print("\nADABOOST REGRESSOR")
print("Mean squared error =", round(mse, 2))
print("Explained variance score =", round(evs, 2))
```

7. This regressor has an inbuilt method that can be called to compute the relative feature importance:

```
# Extract feature importances
feature_importances = regressor.feature_importances_
feature_names = data.columns
```

8. Normalize the values of the relative feature importance:

```
# Normalize the importance values
feature_importances = 100.0 * (feature_importances /
max(feature_importances))
```

9. Sort them so that they can be plotted:

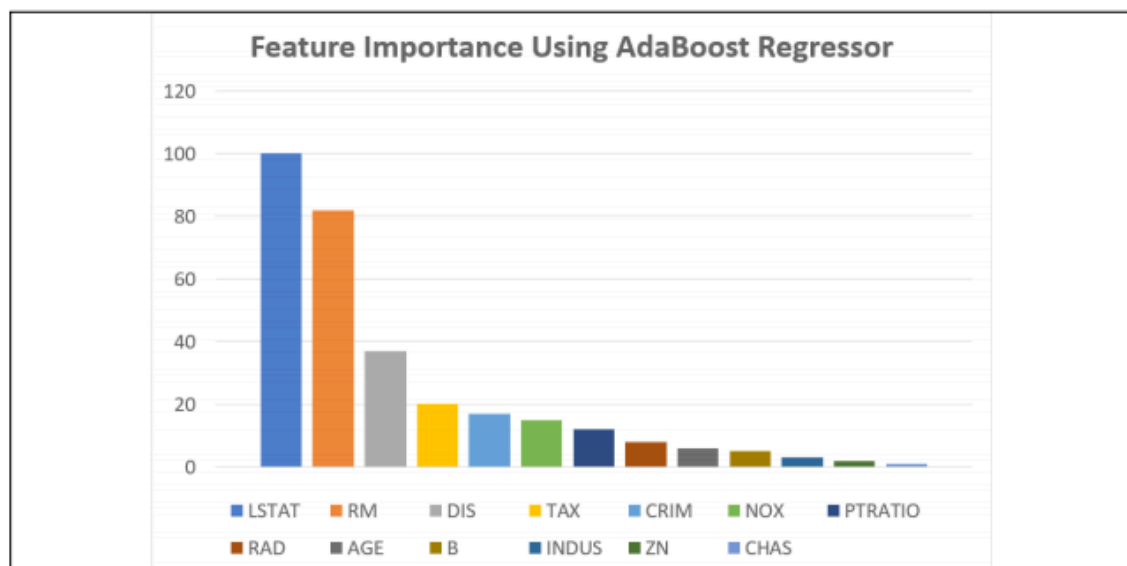
```
# Sort the values and flip them
index_sorted = np.flipud(np.argsort(feature_importances))
```

10. Arrange the ticks on the x axis for the bar graph:

```
# Arrange the X ticks
pos = np.arange(index_sorted.shape[0]) + 0.5
```

9. Use **plt.figure()** to plot the graph.

If successful, it should be like this:





Workshop 4.6 : Construct `feature_importance.py`

1. Combine the code fragment of the above and construct the complete **`feature_importance.py`** Python program.
2. Run the code (if successful) you should see the above chart.
3. Save and submit the chart.
4. What is the TOP 4 feature attributes? Why?



4.7 Mini-project : Predicting traffic using an extremely random forest regressor

Let's apply the concepts learned in the previous sections to a real-world problem. A dataset available at will be used: 'traffic_data.txt'.

This dataset consists of data that counts the number of vehicles passing by on the road during baseball games played at Los Angeles Dodgers stadium. In order to make the data readily available for analysis, we need to pre-process it.

The pre-processed data is in the file `traffic_data.txt`. In this file, each line contains comma-separated strings. Let's take the first line as an example:

Tuesday,00:00,San Francisco,no,3

With reference to the preceding line, it is formatted as follows:

Day of the week, time of the day, opponent team, binary value indicating whether a baseball game is currently going on (yes/no), number of vehicles passing by.

Our goal is to predict the number of vehicles going by using the given information. Since the output variable is continuous valued, we need to build a regressor that can predict the output. We will be using extremely random forests to build this regressor. Let's go ahead and see how to do that.

If your program works: your result should be like that:

Test datapoint: = ['Saturday', '10:20', 'Atlanta', 'no']

Mean absolute error: 7.42

Predicted traffic: 6

Or you allow people to input any Data, Time, Location and the system will predict the traffic at that moment for you.

.....

Summary

In this workshop, we learned about ensemble learning and how it can be used in the real world. We discussed decision trees and how to build a classifier based on it.

We learned about random forests and extremely random forests, which are created from ensembling multiple decision trees. We discussed how to build classifiers based on them. We understood how to estimate the confidence measure of the predictions.

We also learned how to deal with the class imbalance problem.

We discussed how to find the most optimal training parameters to build the models using grid search. We learned how to compute relative feature importance.

We then applied ensemble learning techniques to a real-world problem, where we predicted traffic using an extremely random forest regressor.

In the next workshop, we will discuss unsupervised learning and how to detect patterns in stock market data.

In []: